ECMA Script 6

Programação Web 2

Objetivo de Aprendizagem

- Revisar a sintaxe padrão ES6

Agenda

- ECMA Script 6
- Sintaxe

ECMASript 6

O que é ES6?

> ECMAScript 6 (ES6/ES2015) é a sexta versão do JavaScript

ES6

Por que?

- Código mais limpo e legível
- Sintaxe compatível com JSX
- Recursos que facilitam a criação de componentes
- **Código React moderno usa ES6**

Sintaxe ES6

Declaração de Variáveis

`var`

> Antes do ES6 `var` era a forma padrão de declaração de variáveis. Deve ser evitado no ES6.

> `var` define um escopo global para uma variável

```
1  for (var i = 0; i < 5; i++) {
2    console.log("Inside the loop:", i);
3  }
4  console.log("Outside the loop:", i);
```

```
1  Inside the loop: 0
2  Inside the loop: 1
3  Inside the loop: 2
4  Inside the loop: 3
5  Inside the loop: 4
6  Outside the loop: 5
```

Declaração de Variáveis

`var`

- Permite que uma variável seja reatribuída

```
1   var counter = 10;
2   var counter;
3   console.log(counter); // 10
```

Declaração de Variáveis

`let`

> Define um escopo de bloco para uma variável. Declaração padrão do ES6.

```
1   let x = 10;
2   if (x == 10) {
3       let x = 20;
4       console.log(x); // 20:  reference x inside the block
5   }
6   console.log(x); // 10: reference at the begining of the script
```

Declaração de Variáveis

`let`

> Não é possível redeclarar uma variável declarada orinalmente com `let`

```
1   let counter = 0;
2   let counter;
3   console.log(counter);
```

```
1   Uncaught SyntaxError: Identifier 'counter' has already been declared
```

Declaração de Variáveis

`const`

> Declara um tipo imutável, apenas de leitura. Não é possível fazer uma reatribuição de valor.

```
1   const RATE = 0.1;
2   RATE = 0.2; // TypeError
```

Declaração de Variáveis

`const`

> A inicialização é obrigatória

```
1   const RED; // SyntaxError
```

Arrow Functions

> AFs foram disponibilizadas no ES6 como uma formato mais compacto para escrita de funções

```
1  let add = function (x, y) {
2    return x + y;
3  };
4
5  console.log(add(10, 20)); // 30
```

```
1  let add = (x, y) ⇒ x + y;
2
3  console.log(add(10, 20)); // 30;
```

Arrow Functions

> Quando blocos `{}` são utilizados, `return` é obrigratório

```
1  let add = (x, y) => {
2    return x + y;
3  };
```

Arrow Functions

Exemplo com React

```
1   // Componente funcional (padrão moderno)
2   const Botao = ({ texto, onClick }) => {
3     return <button onClick={onClick}>{texto}</button>
4   }
5   // Callback inline
6   <button onClick={() => console.log('clicou')}>
7     Clique
8   </button>
9   // Mapeando listas
10  const Lista = ({ itens }) => (
11    <ul>
12      {itens.map(item => (
13        <li key={item.id}>{item.nome}</li>
14      ))}
15    </ul>
16  )
```

Template Strings

Exemplo

```
1  const nome = "Maria"
2  const idade = 28
3
4  // Antes (concatenação)
5  console.log("Meu nome é " + nome + " e tenho " + idade + " anos.")
6
7  // Com template string (ES6)
8  console.log(`Meu nome é ${nome} e tenho ${idade} anos.`)
```

Template Strings

Exemplo com React

```
1   const Card = ({ titulo, descricao }) => {
2     // Classes dinâmicas
3     const classeCSS = `card card-${titulo.toLowerCase()}`
4
5     // Conteúdo dinâmico
6     return (
7       <div className={classeCSS}>
8         <h3>{`Título: ${titulo}`}</h3>
9         <p>{descricao}</p>
10        <span>{`Última atualização: ${new Date().toLocaleDateString()}`}</span>
11      </div>
12    )
13
14  // CSS-in-JS com template strings
15  const estilo = `
16    background: linear-gradient(45deg, ${cor1}, ${cor2});
17    padding: ${espacamento}px;
18  `
```

Destructuring

Desestruturação

> Extrair valores de objetos e arrays de forma elegante

```
1  // Sem destructuring
2  const Usuario = (props) => {
3    return <h1>{props.nome} - {props.email}</h1>
4  }
5
6  // Com destructuring (mais comum em React)
7  const Usuario = ({ nome, email }) => {
8    return <h1>{nome} - {email}</h1>
9  }
```

Destructuring

Exemplo

```
1   const Exemplo = () => {
2     // Destructuring em hooks
3     const [usuario, setUsuario] = useState(null)
4
5     // Destructuring em objetos retornados
6     useEffect(() => {
7       fetch('/api/usuario')
8         .then(res => res.json())
9         .then(({ nome, email, endereco: { cidade } }) => {
10          // Extraímos nome, email e cidade diretamente
11          console.log(cidade) // 'São Paulo'
12          setUsuario({ nome, email })
13        })
14    }, [])
15
16    return <div>{/* ... */}</div>
17  }
```

Operador Spread

...

> "Espalha" elementos de arrays/objetos ou cria cópias

```
1  const [form, setForm] = useState({ nome: '', email: '', idade: '' })
2
3  // Atualizando apenas um campo
4  const handleChange = (campo, valor) => {
5    setForm({
6      ...form,        // Mantém todos os campos existentes
7      [campo]: valor  // Sobrescreve apenas o campo específico
8    })
9  }
```

Combinando Objetos com Spread

```
1   // Combinando configurações
2   const defaults = { theme: 'light', lang: 'pt-BR' }
3   const userConfig = { theme: 'dark', notifications: true }
4
5   const config = {
6     ...defaults,     // { theme: 'light', lang: 'pt-BR' }
7     ...userConfig    // Sobrescreve theme, adiciona notifications
8   }
9   // Resultado: { theme: 'dark', lang: 'pt-BR', notifications: true }
10
11  // Passando props com spread
12  const usuario = {
13    nome: 'Ana',
14    email: 'ana@email.com',
15    idade: 25
16  }
17
18  // Passa todas as propriedades como props individuais
19  <Usuario {...usuario} />
```

Spread com Arrays

```
1   const ListaTarefas = () => {
2     const [tarefas, setTarefas] = useState(['Estudar', 'Codar'])
3
4     const adicionarTarefa = (novaTarefa) => {
5       // Adiciona no final
6       setTarefas([...tarefas, novaTarefa])
7
8       // Ou adiciona no início
9       setTarefas([novaTarefa, ...tarefas])
10    }
11
12    const removerTarefa = (index) => {
13      // Remove item específico
14      setTarefas([
15        ...tarefas.slice(0, index),
16        ...tarefas.slice(index + 1)
17      ])
18    }
19
20    return (/* ... */)
21  }
```

Ternary Operator (Operador Ternário)

> Substituto do `if/else` em JSX e lógica simples

```
1   condicao ? valorSeVerdadeiro : valorSeFalso
```

Ternary Operator

Exemplo (renderização condicional)

```
1  const Saudacao = ({ usuario }) => (
2    <div>
3      {usuario ? (
4        <h1>Bem-vindo de volta, {usuario.nome}!</h1>
5      ) : (
6        <button>Fazer login</button>
7      )}
8    </div>
9  )
```

Ternary Operator

Exemplo com React

```
1   // Classes condicionais
2   const BotaoEstado = ({ ativo }) => (
3     <button className={`botao ${ativo ? 'ativo' : 'inativo'}`}>
4       {ativo ? 'Ativo' : 'Inativo'}
5     </button>
6   )
7   // Renderização condicional de componentes
8   <>
9     {isLoading ? (
10      <Spinner />
11    ) : (
12      <Conteudo dados={dados} />
13    )}
14  </>
15  // Lista vazia
16  const Lista = ({ itens }) => (
17    <div>
18      {itens.length > 0
19        ? itens.map(item => <Item key={item.id} {...item} />)
20        : <p>Nenhum item encontrado</p>
21      }
```

ES6 Modules

> Sistema de módulos nativo do JavaScript

```javascript
// utils/formatadores.js

// Export nomeado (múltiplos por arquivo)
export const formatarMoeda = (valor) => {
  return `R$ ${valor.toFixed(2)}`
}

// Export default (apenas um por arquivo)
export default {
  moeda: formatarMoeda,
  data: formatarData
}
```

ES6 Modules

import

```
1  // Componente.jsx
2  // Import nomeado (com chaves)
3  import { formatarMoeda, formatarData } from './utils/formatadores'
4  // Import default (sem chaves)
5  import formatadores from './utils/formatadores'
6  // Renomeando imports
7  import { formatarMoeda as moeda } from './utils/formatadores'
8  // Importando tudo
9  import * as Formatadores from './utils/formatadores'
```

ES6 Modules

Exemplo com React

```
1   // components/Botao.jsx
2   export const Botao = ({ children, onClick }) => (
3     <button onClick={onClick}>{children}</button>
4   )
5
6   // components/index.js - Barrel file
7   export { Botao } from './Botao'
8   export { Card } from './Card'
9   export { Input } from './Input'
10
11  // App.jsx - importando múltiplos componentes
12  import { Botao, Card, Input } from './components'
```

Promises

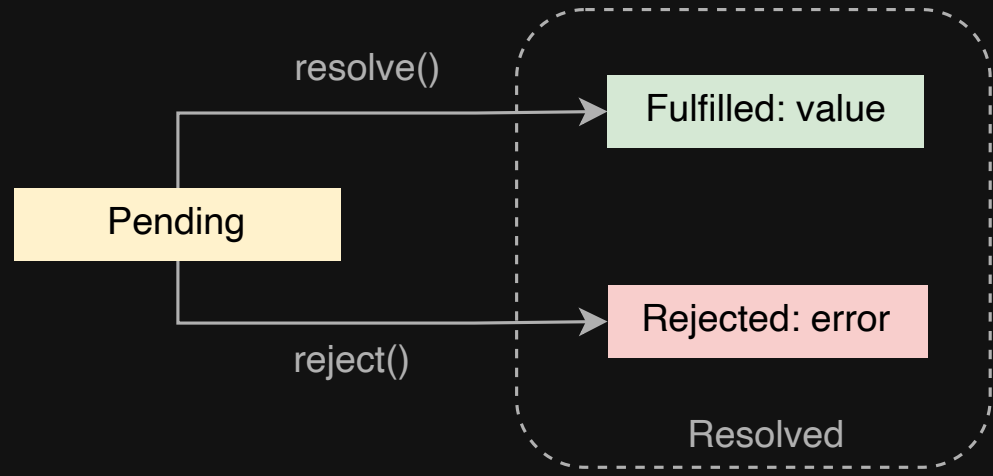
> ES6 define uma `Promise` como sendo um objeto que encapsula o resultado de uma operação assíncrona.

Promises

Estados

Um objeto `promise` pode assumir os estados:

- *Pending*
- *Fulfilled* junto com um valor
- *Rejected* junto com um motivo



Perguntas

Exercícios

1

Realizar o Quiz ES6 disponível na Aula 4 do Google Classroom.

Referências

- ES6 Tutorial
- React

José Roberto Bezerra

jbroberto@ifce.edu.br

jbroberto76

pw2-repo

Powered by Slidev

Cover image by haikei